HANDS-ON RAG PIPELINE

RAG in 4 Hours

Learning Journal

What I built, what I learned, and what comes next

Ashish Gayakwar

MacBook M1 Pro 16GB | Python 3.13 | ChromaDB | Ollama | GPT-4o

Table of Contents

01 What is RAG — The Concept I Learned First **03**

02 The 6 Concepts I Understood Before Building **04**

↳ Concepts 4–6 (continued) **05**

03 What I Installed and Set Up **06**

04 The Files I Built — Structure & Indexing **07**

↳ The Files I Built — Query Pipeline **08**

05 The Experiments I Ran — Chunks & Size **09**

↳ The Retrieval Problem & Stage Comparison **10**

06 Key Lessons I Learned **11**

07 What Comes Next — Stage 3 & Closing **12**

08 How to Pick Up Where I Left Off **13**

13 Pages · 8 Sections · 3 Experiments · 8 Key Lessons

STAGE 1 & 2 COMPLETE

What is RAG — The Concept I Learned First

RAG stands for **Retrieval-Augmented Generation**. Before writing a single line of code, I understood the core idea:

An AI model like GPT-4o knows a lot of general things. But it doesn't know **my documents**. Instead of giving it the whole document every time (expensive, slow, impossible for large files), RAG does two things:

FIND

Search the documents and pull out only the **2–5 most relevant paragraphs** for this specific question.

ANSWER

Give those paragraphs + the question to the AI. It reads them and **answers from them only**.

📌 The AI is not doing the retrieval. The **embedding model + vector database** does the retrieval. The AI only reads what retrieval hands it. This is the most important mental model in RAG.

The RAG Pipeline at a Glance

📄 **PDF** Raw files → 📝 **Text** Extracted → ✂️ **Chunks** Split → 🔢 **Vectors** Embedded → 🗄️ **DB** Stored → 🔍 **Search** Retrieved → 🤖 **LLM** Generated

The 6 Concepts I Understood Before Building

I chose to understand all 6 concepts first, then build. Here is what each one means:

1 CONCEPT

Your PDF Is Just Text

A PDF looks like a document to me. To my Mac it's just a file it can display. Before anything intelligent can happen, the **raw text needs to be extracted** — every word, every paragraph, every page — as plain text Python can work with. A tool called **pypdf** does this.

2 CONCEPT

You Can't Feed the Whole PDF to the AI

Every AI model has a limit on how much text it can read at once (called a **context window**). So the extracted text gets cut into small pieces called **chunks**. Each chunk is roughly a paragraph or half a page. Think of cutting a book into index cards — each card has one self-contained idea.

3 CONCEPT

Turning Words into Numbers (Embeddings)

Computers don't understand words — they understand numbers. Every chunk gets converted into a list of numbers called an **embedding** or **vector**. The key property: chunks that mean similar things get similar numbers, even if they use different words. This is what allows the system to find relevant content even when the question uses different words than the document.

4
CONCEPT

Storing Those Numbers (Vector Database)

All the chunks and their number lists need to be stored somewhere that allows **very fast searching**. A vector database (ChromaDB in my case) stores them and can instantly find the chunks whose numbers are closest to any given query. It saves everything to disk so it persists between runs.

5
CONCEPT

Finding the Right Chunks (Retrieval)

When I ask a question, it goes through the same embedding process — converted to numbers. ChromaDB finds the chunks whose numbers are closest. Those are the **most relevant chunks**. No keyword matching — pure **meaning-based search**.

6
CONCEPT

Letting the AI Answer (Generation)

The retrieved chunks + my question get sent to the AI model with a strict instruction: **answer only from this context**. The AI reads those specific paragraphs and forms a clear, grounded answer. It cannot make things up because it's constrained to what retrieval handed it.

The Complete Flow

Extract (pypdf) → **Chunk** (1000 chars) → **Embed** (nomic-embed-text) → **Store** (ChromaDB) → **Retrieve** (top 5) → **Generate** (LLM answer)

What I Installed and Set Up

My Machine

MacBook M1 Pro

16GB RAM — M1 chip is excellent for local AI models

VS Code

Code editor and terminal in one place

Python 3.13.3

Already on Mac. Verified with python3 --version

The Setup Steps I Ran

- **Created project folder** — my-rag-project
- **Created virtual environment** — python3 -m venv venv
- **Activated it** — source venv/bin/activate — the (venv) prefix confirms it's active
- **Installed all libraries** — pip install pypdf chromadb sentence-transformers einops ollama openai python-dotenv
- **Downloaded Ollama app** — from ollama.com — installed as a Mac app, runs as a menu bar icon
- **Downloaded AI model** — ollama pull llama3.1:8b — 4.7GB, best quality for 16GB RAM
- **Downloaded embedding model** — ollama pull nomic-embed-text — 274MB

```
# Quick verification commands
python3 --version          # Python 3.13.3
ollama list                # shows llama3.1:8b + nomic-embed-text
pip list | grep chromadb   # confirms chromadb installed
```

The Files I Built

Project Folder Structure

```
my-rag-project/
├── venv/                    ← Python virtual environment
├── docs/                    ← My 6 PDF files live here
│   ├── 01. FinalAIEcom.pdf
│   ├── 02. AI Video.pdf
│   ├── 03. AI Job Final.pdf
│   ├── 04. AI Market Final.pdf
│   ├── 05. AI Fintech Updated.pdf
│   └── 06. AI HR.pdf
├── my_vector_db/            ← ChromaDB database (auto-created)
├── index.py                 ← Run once to index PDFs
├── query.py                 ← Run every time to ask questions
└── .env                     ← API key stored safely here
```

index.py — What It Does

This script runs **once**. It reads all PDFs, splits them into chunks, embeds them, and stores everything in ChromaDB. I only need to run it again if I add new PDFs.

`load_pdfs()`	Reads every PDF in docs/ folder, extracts text page by page, tracks which file and page each chunk came from
`split_into_chunks()`	Cuts each page into 1000-character chunks with 50-character overlap so answers don't get cut at chunk boundaries
`index_documents()`	Embeds every chunk using nomic-embed-text and stores in ChromaDB with source metadata

> 📌 The script asks **'Clear existing index and rebuild? (yes/no)'** before wiping anything. This prevents accidentally destroying a good index.

query.py — What It Does

This script runs **every time I have a question**. It loads fast because all the heavy work (embedding, indexing) is already done. It runs in a loop so I can ask multiple questions without restarting.

1. **Checks database** — If nothing is indexed, stops and tells me to run index.py first
2. **Checks API key** — Reads from .env file, stops with a clear message if missing
3. **Embeds the question** — Converts my question to numbers using the same embedding model
4. **Searches ChromaDB** — Retrieves top 5 most relevant chunks
5. **Calls GPT-4o** — Sends chunks + question, gets a detailed answer
6. **Shows sources** — Prints which PDF files the answer came from
7. **Error handling** — If OpenAI call fails, shows a clear message and continues the loop

Two-Script Architecture

`index.py`

Run **once** (or when PDFs change). Does the heavy work: extract, chunk, embed, store. Takes minutes.

→

`query.py`

Run **every time**. Loads fast, runs in a loop. Retrieve + generate only. Takes seconds.

📌 Splitting into index.py and query.py was a real architectural improvement. The database persists between runs — no need to re-process PDFs every time.

The Experiments I Ran

After the basic system worked, I ran three experiments to understand how each variable affects answer quality.

Experiment 1 — Chunks Retrieved (n_results)

I tested asking the same question with 3, 5, and 6 chunks retrieved:

n_results	Context Sent	Answer Quality	Verdict
3	Limited — might miss key info	Sometimes incomplete	Too few
5	Good coverage across PDFs	Detailed, well-sourced	✅Sweet spot
6	Extra chunk adds noise	Slightly more verbose	Diminishing returns

✅Settled on **n_results=5** as the sweet spot for my 6 PDFs.

Experiment 2 — Chunk Size

I tested three different chunk sizes on the same question about AI in payments and fraud detection:

Chunk Size	Chunks Created	Retrieval Quality	Answer Quality
500 chars	Many small chunks	Fragments — loses paragraph coherence	Choppy, incomplete
1000 chars	Balanced count	Preserves complete paragraphs	✅Clear, detailed
2000 chars	Fewer large chunks	Too much irrelevant text per chunk	Verbose, unfocused

Experiment 3 — The Retrieval Problem

I asked **"What is the impact of AI on FinTech?"** and got a wrong answer — the system said it couldn't find anything even though I have a FinTech PDF.

I then printed the retrieved chunks and discovered:

- ✗ The 3 chunks retrieved were all from the Jobs PDF and author bio — nothing from the FinTech PDF
- ✗ The word 'FinTech' didn't match the specific language in the PDF well enough

I then asked **"What is the impact of AI on payments and fraud detection?"** — much more specific — and instantly got the right chunks from the right PDF with a detailed answer.

> 📌 **The most important lesson:** retrieval quality depends on how you ask the question, not just what's in the documents. Specific questions find the right chunks. Broad questions miss them.

Stage 1 vs Stage 2 — What Changed

	Stage 1 — Local	Stage 2 — Cloud
LLM	llama3.1:8b (Ollama)	GPT-4o (OpenAI API)
Embedding	nomic-embed-text (local)	nomic-embed-text (local)
Vector DB	ChromaDB (local)	ChromaDB (local)
Cost	Free — runs on Mac	$0.01–0.03 per query
Quality	Good for simple questions	Excellent — detailed answers
Speed	~5–10s per answer	~2–3s per answer
Privacy	Everything stays on Mac	Question + chunks sent to OpenAI

Key Lessons I Learned

1. **Retrieval is everything.** The LLM is only as good as the chunks it receives. If retrieval fails, the answer fails — no matter how smart the model. This is the core insight of RAG.

2. **Chunk size matters.** Too small = fragmented meaning. Too large = irrelevant noise included. 1000 characters worked best for my well-structured PDFs.

3. **Specific questions work better than broad ones.** "Impact of AI on FinTech" failed. "Impact of AI on payments and fraud detection" succeeded. The embedding matches meaning — but meaning needs enough specificity to find the right content.

4. **Index once, query forever.** Splitting into index.py and query.py was a real architectural improvement. The database persists between runs — no need to re-process PDFs every time.

5. **Local and cloud are interchangeable.** Switching from llama3.1:8b to GPT-4o was 10 lines of code. The pipeline doesn't care which model generates the answer.

6. **Source tracking builds trust.** Showing which PDF the answer came from makes the system trustworthy. Users can verify answers against the original document.

7. **Security matters even for personal projects.** Storing the API key in .env instead of in the code is a habit worth building from day one.

8. **Code reviews find real bugs.** Running Codex reviews on the code revealed real issues — stale vectors, unstable IDs, missing error handling. Fixing them made the system more robust.

What Comes Next — Stage 3

Stage 3 has three parts. I understand what each one means:

3A

Chat UI (Streamlit)

Right now I use the terminal. Stage 3A gives me a proper chat window in the browser — like ChatGPT. Same RAG system underneath, just a proper interface on top. **Tool: Streamlit. Estimated time: 1–2 hours.**

3B

Conversation Memory

Right now every question is independent. If I ask "what is AI in fintech?" then "what are the risks?" — the system doesn't know "risks" refers to fintech. Stage 3B adds **memory so follow-up questions work naturally** within a session.

3C

LangChain (Separate Folder)

LangChain is a framework that provides pre-built pieces for RAG pipelines so you write less code. I will build this in a separate folder called **langchain-rag** so my working system stays untouched. Since I built everything manually first, I'll understand exactly what LangChain is abstracting.

📌 **LangGraph** is a newer tool from the same team as LangChain. It handles AI agents that make decisions — do step 1, then decide whether to do step 2 or step 3. More advanced, relevant when building agents rather than simple pipelines.

PROGRESS TRACKER

✅ Stage 1	✅ Stage 2	Stage 3A	Stage 3B	Stage 3C
Local LLM	Cloud LLM	Chat UI	Memory	LangChain

How to Pick Up Where I Left Off

Everything is saved. When I come back:

1. Open VS Code
2. Open the terminal inside VS Code (Control + `)
3. `source venv/bin/activate`
4. `python3 query.py`
5. Ask questions — the database is still there, nothing needs rebuilding

⚠️ Only run **python3 index.py** again if you add new PDFs to the docs/ folder. It will ask if you want to clear the old index first.

Built from zero.
Understood every step.
Ready for Stage 3.

Ashish Gayakwar · February 2026